Convolutional Neural Networks

Conv variants, parameter sharing, attention, ResNet

Moin Mostakim

Learning goals

After this lecture you can:

1. Contrast an MLP on pixels with a CNN (local connectivity + sharing).
2. Distinguish **convolution** from **cross-correlation** and say which libraries use.
3. Compute a convolution **by hand** on 5×5 and 6×6 maps; track 2-D shapes through conv$\rightarrow$pool$\rightarrow$conv$\rightarrow$pool.
4. Derive **backprop** through a conv layer; define receptive field / hierarchy.
5. Design a small CNN; explain ResNet; compare nets by params / FLOPs / accuracy.
6. Name conv **variants** (depthwise, dilated, grouped, ...) and **attention** blocks (SE, CBAM, ...).

MLP recap — one dense layer

A fully connected layer:

$$\mathbf{y} = \phi(\mathbf{W}\mathbf{x} + \mathbf{b}), \qquad \mathbf{W} \in \mathbb{R}^{n_{\text{out}} \times n_{\text{in}}}$$

Every output unit sees **every** input. Number of weights $= n_{\text{out}}\, n_{\text{in}}$.

Strength

Any pairing of features. Works for unstructured vectors.

On images

Ignores locality. Same object shifted needs new weights. Too many parameters.

How images enter a network

An RGB image is a 3-D **tensor**, not a flat vector:

$$\mathbf{X} \in \mathbb{R}^{H \times W \times C} \quad (H = \text{height},\ W = \text{width},\ C = \text{channels})$$

Example: 32×32 RGB $\Rightarrow 32 \times 32 \times 3 = 3072$ numbers.

MLP flattening

Stack into $\mathbf{x} \in \mathbb{R}^{3072}$. Nearby pixels become unrelated indices.

CNN keeps layout

Kernels slide over (h, w). Channel axis is the feature dimension.

Inductive biases we want: nearby pixels interact; the same pattern may appear anywhere.

From MLP to CNN — three design choices

1. **Local connectivity**: an output depends only on a $k \times k$ patch, not the whole image.
2. **Parameter sharing**: the same $k \times k$ weights are reused at every location (translation equivariance).
3. **Pooling / stride**: downsample spatially; fewer units in later stages (and we can shrink or drop FC layers).

CNN layer = local MLP patch + tied weights + (optional downsample)

Components of a convolution operation

A conv layer is more than "multiply a filter." Pieces:

- **Input tensor** $H_{\text{in}} \times W_{\text{in}} \times C_{\text{in}}$
- **Kernel / filter** $k_h \times k_w \times C_{\text{in}} \times C_{\text{out}}$
- **Bias** one per output channel
- **Stride** s: step of the window
- **Padding** p: zeros (or reflect) at borders
- **Dilation** d: holes in the kernel
- **Nonlinearity** ReLU / GELU after the sum
- Optional **BN**, **pooling**, residual add

At one location: take a patch $\rightarrow$ multiply with kernel (per input channel) $\rightarrow$ **sum** $\rightarrow$ add bias $\rightarrow$ that is **one** output pixel of **one** output channel. Repeat for every location and every output channel.

Convolution vs cross-correlation



Mathematical convolution **flips** the kernel; cross-correlation does **not**.

Convolution

$(f * k)[i, j] = \sum_{u,v} f[i - u, j - v]\, k[u, v]$

Kernel is rotated 180° first.

Cross-correlation

$(f \star k)[i, j] = \sum_{u,v} f[i + u, j + v]\, k[u, v]$

Window as you wrote the filter.

Why DL uses correlation. Learned k can absorb a flip. PyTorch `conv2d` and TensorFlow `conv2d` compute **cross-correlation**. We still say "convolution layer."

If $k' = \mathrm{flip}(k)$, then $f * k = f \star k'$. Training never needs the extra flip.

Why use this operator at all?

- **Local statistics**: edges, textures live in small neighborhoods.
- **Stationarity**: a vertical edge detector should work in every corner of the photo.
- **Composition**: stack layers $\Rightarrow$ large receptive field from tiny kernels.
- **Fewer parameters** than tying an MLP to every pixel pair.

Convolution / correlation is a **linear filter bank** with tied weights. Nonlinearity after it lets the net detect "edge," then "corner," then "object."

Manual convolution (cross-correlation)

Input X (3×3), kernel K (2×2), stride 1, no padding:

$$X = \begin{bmatrix} 1 & 2 & 0 \\ 1 & 0 & 1 \\ 2 & 1 & 1 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

Top-left patch: $1 \cdot 1 + 2 \cdot 0 + 1 \cdot 0 + 0 \cdot (-1) = \mathbf{1}$.
Top-right: $2 \cdot 1 + 0 \cdot 0 + 0 \cdot 0 + 1 \cdot (-1) = \mathbf{1}$.
Bottom-left: $1 \cdot 1 + 0 \cdot 0 + 2 \cdot 0 + 1 \cdot (-1) = \mathbf{0}$.
Bottom-right: $0 \cdot 1 + 1 \cdot 0 + 1 \cdot 0 + 1 \cdot (-1) = -\mathbf{1}$.

$$Y = \begin{bmatrix} 1 & 1 \\ 0 & -1 \end{bmatrix} \in \mathbb{R}^{2 \times 2}$$

Output tensor dimensions

For square kernel k, stride s, padding p, dilation 1:

$$H_{\text{out}} \;=\; \left\lfloor \frac{H_{\text{in}} + 2p - k}{s} \right\rfloor + 1 \quad \text{(same formula for } W_{\text{out}}\text{)}$$

Example. $H_{\text{in}} = W_{\text{in}} = 32$, $k = 5$, $p = 2$ ("same"), $s = 1$:

$$H_{\text{out}} = \lfloor (32 + 4 - 5)/1 \rfloor + 1 = 32.$$

With $C_{\text{in}} = 3$, $C_{\text{out}} = 16$, batch $N = 8$:

$$\text{output shape} \;=\; N \times 16 \times 32 \times 32.$$

Valid conv: $p = 0$. "Same" for odd k: $p = (k-1)/2$.

This lecture: $k = 3$, $s = 1$, $p = 0 \Rightarrow 5 \times 5 \to 3 \times 3$ and $6 \times 6 \to 4 \times 4$.

Simulation setup — 2D maps

Same kernel on two images. Stride 1, **no padding** (valid). Bias 0.

1	2	0	1	3
0	1	2	0	1
2	0	1	3	0
1	2	0	1	2
0	1	3	0	1

$X_5 \in \mathbb{R}^{5\times5}$

1	0	0
0	1	0
0	0	1

$K \in \mathbb{R}^{3\times3}$

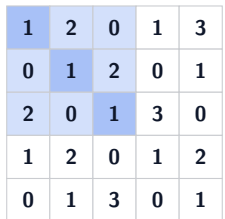
1	2	0	1	3	1
0	1	2	0	1	2
2	0	1	3	0	1
1	2	0	1	2	0
0	1	3	0	1	2
2	0	1	2	0	1

$X_6 \in \mathbb{R}^{6\times6}$

Why this K? Only the **main diagonal** is 1. So $y_{ij} = x_{ij} + x_{i+1,j+1} + x_{i+2,j+2}$. Check the three shaded taps on any 3×3 window — no other multiplies.

5×5 — 2D shape of the slide

3×3 window, stride 1: it can sit in 3 row starts $\times$ 3 col starts $\Rightarrow$ output is a 3×3 matrix.



Window at $(0, 0)$ on X_5

$\longrightarrow$

$y_{00} = 1 + 1 + 1$



$Y \in \mathbb{R}^{3\times3}$ filling

Shift the window **one cell** right $\rightarrow$ write y_{01}. After 3 steps, next **row** of Y. Never wrap; the last valid top-left is $(2, 2)$.

5×5 — **window moves, Y fills**

origin $(0, 0)$

1	**2**	**0**	1	3
0	**1**	**2**	0	1
2	**0**	**1**	3	0
1	2	0	1	2
0	1	3	0	1

$1 + 1 + 1 = \mathbf{3}$

origin $(0, 1)$

1	**2**	**0**	**1**	3
0	**1**	**2**	**0**	1
2	**0**	**1**	**3**	0
1	2	0	1	2
0	1	3	0	1

$2 + 2 + 3 = \mathbf{7}$

origin $(0, 2)$

1	2	**0**	**1**	**3**
0	1	**2**	**0**	**1**
2	0	**1**	**3**	**0**
1	2	0	1	2
0	1	3	0	1

$0 + 0 + 0 = \mathbf{0}$

First row of Y:

3	**7**	**0**

$5 \times 5 \star 3 \times 3$ — every output cell

$y_{00} = 1+1+1 = \mathbf{3}$ $y_{01} = 2+2+3 = \mathbf{7}$ $y_{02} = 0+0+0 = \mathbf{0}$
$y_{10} = 0+0+0 = \mathbf{0}$ $y_{11} = 1+1+1 = \mathbf{3}$ $y_{12} = 2+3+2 = \mathbf{7}$
$y_{20} = 2+2+3 = \mathbf{7}$ $y_{21} = 0+0+0 = \mathbf{0}$ $y_{22} = 1+1+1 = \mathbf{3}$

1	**2**	**0**	**1**	**3**
0	**1**	**2**	**0**	**1**
2	**0**	**1**	**3**	**0**
1	**2**	**0**	**1**	**2**
0	**1**	**3**	**0**	**1**

input 5×5

$\rightarrow$

3	**7**	**0**
0	**3**	**7**
7	**0**	**3**

output 3×3

6×6 — 2D shape of the slide

$6 - 3 + 1 = 4$ positions per axis $\Rightarrow Y \in \mathbb{R}^{4\times 4}$ (sixteen windows).

1	**2**	**0**	**1**	**3**	**1**
0	**1**	**2**	**0**	**1**	**2**
2	**0**	**1**	**3**	**0**	**1**
1	**2**	**0**	**1**	**2**	**0**
0	**1**	**3**	**0**	**1**	**2**
2	**0**	**1**	**2**	**0**	**1**

first window on X_6; $y_{00} = 1 + 1 + 1 = 3$

3	·	·	·
·	·	·	·
·	·	·	·
·	·	·	·

output canvas 4×4

$6 \times 6 \star 3 \times 3$ — **full** 4×4

1	**2**	**0**	**1**	**3**	**1**
0	**1**	**2**	**0**	**1**	**2**
2	**0**	**1**	**3**	**0**	**1**
1	**2**	**0**	**1**	**2**	**0**
0	**1**	**3**	**0**	**1**	**2**
2	**0**	**1**	**2**	**0**	**1**

6×6

$\rightarrow$

3	**7**	**0**	**3**
0	**3**	**7**	**0**
7	**0**	**3**	**8**
3	**7**	**0**	**3**

4×4

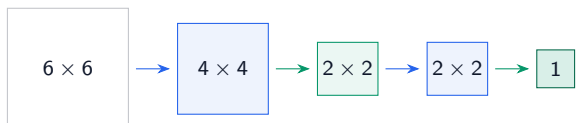
Extra last row/col of X_6 is why we get 4 not 3: one more valid origin.

Stack: conv → max-pool → conv → max-pool

Start from the 6×6 simulation. Keep **2-D layout** at every layer (never flatten).

$$6 \times 6 \xrightarrow{\text{conv } 3\times3,\, p=0} 4 \times 4 \xrightarrow{\text{pool } 2\times2,\, s=2} 2 \times 2$$

$$2 \times 2 \xrightarrow{\text{conv } 3\times3,\, p=1} 2 \times 2 \xrightarrow{\text{pool } 2\times2,\, s=2} 1 \times 1$$



2×2 is too small for a valid 3×3 ($2 - 3 + 1 < 1$). Second conv uses **padding** 1: zero border → 4×4 padded map → $4 - 3 + 1 = 2$ again.

Same K reused so you can audit every number. A real net would learn a new K_2.

After conv_1: 4×4 and the four pool windows

Non-overlapping 2×2 max-pool, stride 2: the 4×4 splits into four quadrants.

3	7	0	3
0	3	7	0
7	0	3	8
3	7	0	3

conv_1 output (four 2×2 blocks)

$$\max(3, 7, 0, 3) = \mathbf{7}$$
$$\max(0, 3, 7, 0) = \mathbf{7}$$
$$\max(7, 0, 3, 7) = \mathbf{7}$$
$$\max(3, 8, 0, 3) = \mathbf{8}$$

7	7
7	8

after max-pool_1: 2×2

After $\mathbf{pool}_1 \rightarrow \mathbf{pad} \rightarrow \mathbf{conv}_2$

2×2 is padded by one zero on each side (2-D view of $p = 1$):

0	0	0	0
0	**7**	**7**	0
0	**7**	**8**	0
0	0	0	0

padded 4×4

TL window: $0 + 7 + 8 = \mathbf{15}$
TR window: $0 + 7 + 0 = \mathbf{7}$
BL window: $0 + 7 + 0 = \mathbf{7}$

BR window: $7 + 8 + 0 = \mathbf{15}$

15	**7**
7	**15**

after $\mathbf{conv}_2$: still 2×2

After $\text{conv}_2 \rightarrow \text{max-pool}_2$

Last pool is one 2×2 window covering the whole map.



$$\max(15, 7, 7, 15) \quad = \quad \mathbf{15}$$

A classifier would now apply a linear layer on this scalar (or on many channels of 1×1 maps).

5×5 through the same stack — shapes only

Valid 3×3 on 5×5 already gave $Y = \begin{bmatrix} 3 & 7 & 0 \\ 0 & 3 & 7 \\ 7 & 0 & 3 \end{bmatrix}$. Odd size meets stride-2 pool:

$$5 \times 5 \xrightarrow{\text{conv } 3\times3,\, p=0} 3 \times 3 \xrightarrow{\text{pool } 2\times2,\, s=2} 1 \times 1$$

Top-left 2×2 of Y: $\max(3, 7, 0, 3) = \mathbf{7}$. The last row and last column of the 3×3 are **dropped** (classic odd-map / even-pool mismatch).

No room for a second 3×3 after that 1×1. That is why the numeric stack used 6×6: it stays even after the first valid conv (4×4), so pool→conv→pool all fit as 2-D matrices.

How parameters are shared

MLP on $32 \times 32 \times 3 \to 100$ hidden units: W has 100×3072 **distinct** weights.
CNN 5×5 kernel, $3 \to 16$ maps: the **same** 25 numbers (times 3 in-channels) are used at **every** of the 32×32 locations.

Share across space

One filter, many positions. Detect the same motif anywhere.

Do not share across channels

Each output channel is its own filter (its own motif).

Sharing $\Rightarrow$ translation **equivariance**: shift the cat, the feature map shifts.

Count CNN parameters

One conv: C_{out} filters, each $k \times k \times C_{\text{in}}$, plus optional bias.

$$\#\text{params} \quad = \quad C_{\text{out}}\big(k \;\cdot\; k \;\cdot\; C_{\text{in}} \;+\; 1_{\text{bias}}\big)$$

Example. $3 \to 16$ maps, 5×5, with bias:

$$16 \cdot (5 \cdot 5 \cdot 3 + 1) = 16 \cdot 76 = \mathbf{1216}$$

MLP to the same $32 \times 32 \times 16$ spatial field as a dense layer from 3072 inputs:

$$32 \cdot 32 \cdot 16 \cdot 3072 \approx \mathbf{50.3} \times \mathbf{10^6}$$

Same "output volume," $\sim 40{,}000\times$ more weights.

Why CNN needs fewer parameters than an MLP

1. **Sparsity**: each output linked to $k^2 C_{\text{in}}$ inputs, not HWC_{in}.
2. **Tying**: those weights are reused at every (i, j).
3. **Depth**: large context from stacking small k, not from a giant dense matrix.

MLP: one global template. CNN: a **dictionary of local templates** scanned over the image. Capacity goes into *which* patterns, not *where*.

Reducing fully connected layers

Classic AlexNet: conv stack $\rightarrow$ flatten $\rightarrow$ huge FC (9216×4096). Most parameters lived in FC.

Replace FC by

Global average pooling (GAP): one number per channel, then a small linear classifier. GoogLeNet / ResNet.

Or 1×1 conv

Mix channels with no spatial explosion. Then GAP. No giant flatten.

1×1 conv on $C_{\text{in}} \rightarrow C_{\text{out}}$: only $C_{\text{in}} C_{\text{out}}$ weights, still shared over space — a cheap "MLP on channels."

Backprop through convolution — scalar view

At one output $y_{i,j} = \sum_{u,v} x_{i+u,j+v}\, k_{u,v}$ (correlation, bias omitted).
Loss L. Need $\partial L/\partial k_{u,v}$ and $\partial L/\partial x_{p,q}$.
Kernel gradient (sum over positions that used that weight):

$$\frac{\partial L}{\partial k_{u,v}} = \sum_{i,j} \frac{\partial L}{\partial y_{i,j}}\, x_{i+u,j+v}$$

This **is** correlation of input X with the incoming gradient map $\delta = \partial L/\partial Y$. Sharing $\Rightarrow$ **accumulate** all locations into one ∂k.

Backprop — input gradient

Chain rule for $x_{p,q}$:

$$\frac{\partial L}{\partial x_{p,q}} = \sum_{u,v} \frac{\partial L}{\partial y_{p-u,q-v}} \, k_{u,v}$$

(indices where the window covered (p, q)). This is convolution / flipped-kernel correlation of δ with K.

Implementation. Forward: $\mathrm{conv}(X, K)$. Backward wrt K: $\mathrm{conv}(X, \delta)$. Backward wrt X: $\mathrm{conv}(\delta, \mathrm{flip}(K))$ with full padding. Same as MLP backprop, but with **tied W**.

Worked 2×2 backprop

From the earlier $Y = \left[\begin{smallmatrix} 1 & 1 \\ 0 & -1 \end{smallmatrix}\right]$, suppose $\delta = \partial L / \partial Y = \left[\begin{smallmatrix} 1 & 0 \\ 0 & 1 \end{smallmatrix}\right]$.
Then $\partial L / \partial k_{00} = x_{00} \cdot 1 + x_{11} \cdot 1 = 1 + 0 = \mathbf{1}$
(and similarly other k entries: correlate X with δ).

Multi-channel: sum over input channels for ∂k; each output channel has its own $K^{(c)}$.
Batch: sum over N.

Receptive field

The **receptive field** of a unit is the input region that can change its value.
One 3×3 conv, stride 1: RF $= 3$. Two stacked 3×3: RF $= 5$. Three: RF $= 7$.

$$\mathrm{RF}_L \;=\; \mathrm{RF}_{L-1} \;+\; (k_L \;-\; 1) \;\cdot\; \textstyle\prod_{\ell=1}^{L-1} s_\ell$$

- Stride / pooling **multiplies** later jumps over the image.
- Dilated conv grows RF without extra params.
- Classification nets aim for RF covering most of the object (or the whole 224×224 crop).

Feature hierarchy

Early
Edges, color blobs. Small RF. High spatial resolution.

Middle
Textures, parts (eye, wheel). Medium RF.

Deep
Object-level. Large RF. Low resolution, many channels.

This hierarchy is why we **stack** convolutions instead of one huge dense layer: composition of local detectors, same as stacking MLP nonlinearities, but on **maps**.

Taxonomy — conv variants

Standard conv mixes **space** and **channels** in one kernel $k \times k \times C_{\text{in}} \times C_{\text{out}}$. Variants **factor** that mix to save compute or grow context.

Family	Idea	Typical use
1×1 / pointwise	channel mixing only	bottleneck, separable
Depthwise	spatial mix per channel	MobileNet
Group	split channels into G groups	ResNeXt, ShuffleNet
Dilated	holes in kernel	segmentation, RF
Deformable	learned offsets	detection
Large-kernel / dynamic	wider or input-dependent K	ConvNeXt, CondConv

1×1 **convolution**

Kernel 1×1: no spatial neighborhood — an **MLP on the channel vector** at each pixel, **shared** over $H \times W$.

$$\#\text{params} \quad = \quad C_{\text{in}} C_{\text{out}} \qquad (+ \text{ bias } C_{\text{out}})$$

- Project $C_{\text{in}} \to C_{\text{out}}$ (bottleneck in ResNet: $256 \to 64 \to 256$).
- Fuse features after a concat (Inception).
- Cheap: no k^2 factor. Output spatial size unchanged if $s = 1$, $p = 0$.

1×1 **is** pointwise convolution — same operator, two names.

Depthwise convolution

One spatial filter **per input channel**; no mixing across channels.

Standard: each of C_{out} outputs sees all C_{in} channels.
Depthwise: $C_{\text{out}} = C_{\text{in}} = C$; kernel shape $k \times k \times 1 \times C$ (plus optional multiplier).

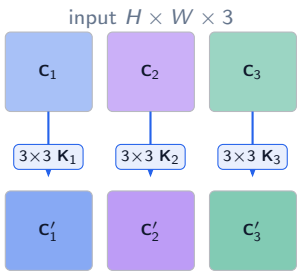
$$\#\text{DW params} \;=\; C \cdot k \cdot k \qquad \text{vs standard } C_{\text{out}} k^2 C_{\text{in}}$$

Example. $C = 64$, $k = 3$: DW $= 576$ weights. Standard $64 \to 64$: 36,864 weights.

Depthwise vs depthwise separable — picture

Depthwise

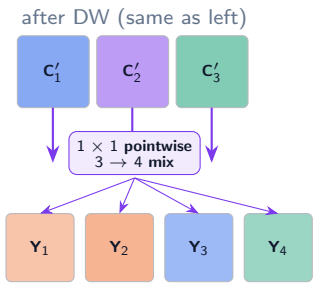
each channel has its *own* 3×3; no mix



output still $H' \times W' \times 3$
channels stay separate

Depthwise separable

DW **then** 1×1 mixes channels

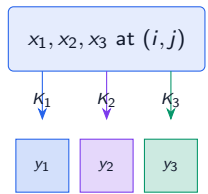


output $H' \times W' \times 4$
each Y sees all C'

Same idea, one spatial location

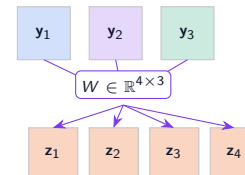
At one pixel, DW is **not** a matrix over channels. Separable *adds* that matrix as a second step.

Depthwise



$y_c = K_c \star x_c$ (own neighborhood)

Then pointwise (separable)



$\mathbf{z} = W\mathbf{y}$ (same W at every pixel)

Read the picture. Depthwise = spatial filter, channels in parallel. Depthwise separable = that, plus a 1×1 MLP that **combines** the filtered maps into new ones (MobileNet).

Pointwise convolution = depthwise separable

Depthwise separable (MobileNet): spatial DW, then pointwise 1×1 to mix channels and set C_{out}.

Cost

$C_{\text{in}} k^2 + C_{\text{in}} C_{\text{out}}$ vs $C_{\text{out}} k^2 C_{\text{in}}$.

Ratio $\approx \frac{1}{C_{\text{out}}} + \frac{1}{k^2}$ (often $\sim$ 8–9$\times$ cheaper for 3×3).

Meaning

DW = "where"; PW = "which combination of maps." Same factorization idea as reducing a dense MLP into smaller factors.

Group convolution

Split C_{in} (and C_{out}) into G groups. Each group is a smaller standard conv; results concatenated.

$$\#\text{params} \quad = \quad \frac{C_{\text{out}}\, k^2\, C_{\text{in}}}{G}$$

- $G = 1$: ordinary conv. $G = C_{\text{in}}$: depthwise.
- **ResNeXt**: grouped 3×3 as "cardinality."
- Isolated groups never talk unless you shuffle / 1×1 later (ShuffleNet).

Dilated / atrous convolution

Insert gaps of size d (dilation) between kernel taps. A 3×3 with $d = 2$ covers a 5×5 neighborhood with still 9 weights.

Effective kernel span: $k_e = d(k-1)+1$. Output size uses k_e in the usual formula.

Why

Grow **receptive field** without pooling (keep resolution) — DeepLab, audio, WaveNet.

Care

Large d can skip pixels ("gridding"). Mix several d (ASPP) or stack increasing d.

Deformable convolution

Standard grid is fixed. **DCN**: predict a 2-D **offset** per kernel tap (extra conv), then sample X at $p_0 + p_n + \Delta p_n$ with bilinear interpolation.

Why. Objects deform, scale, and rotate. Learned sampling adapts the receptive field **shape** per location (detection, segmentation).

More params/FLOPs than plain conv; offsets must be regularized by data — not a free lunch on tiny sets.

Large-kernel convolution

VGG stacked 3×3 because 7×7 was expensive and overfit. After BN/ResNet/modern training, **ConvNeXt / RepLKNet** revisit 7×7 (even 31×31 with depthwise).

- Larger kernel $\approx$ larger RF in **one** layer (fewer sequential 3×3).
- Cost: params $\propto k^2$ unless depthwise: then $\propto Ck^2$ only.
- Often paired with 1×1 channel mix (same recipe as separable).

Dynamic convolution

Kernel is **not** a fixed K; it depends on the input (or on attention over experts).
CondConv / DY-Conv: $K(x) = \sum_{m=1}^{M} \pi_m(x)\, K_m$, with $\pi = \mathrm{softmax}(\mathrm{GAP}(x))$.

Gain

Extra capacity almost for free at inference if M is small: one **aggregated** kernel, then one conv.

Link to MLP

Same idea as a mixture of experts: shared spatial operator, input-dependent combination.

Attention in CNNs — idea

Convolution is **content-unaware** (same K everywhere). Attention **reweights** features using the data.

Channel attention

Which maps matter? Scalar $s_c \in [0,1]$ per channel, $x'_c = s_c\, x_c$.

Spatial attention

Where in $H \times W$? Map $M \in [0,1]^{H \times W}$, $x' = M \odot x$ (broadcast on C).

Cheap if s or M come from pooling + tiny MLP/7×7 conv — not full self-attention $O((HW)^2)$ unless you mean ViT.

Squeeze-and-Excitation (SE)

Hu et al.: squeeze global info, excite per-channel gates.

1. **Squeeze:** GAP $\rightarrow \mathbf{z} \in \mathbb{R}^C$
2. **Excitation:** $\mathbf{s} = \sigma\big(W_2\,\mathrm{ReLU}(W_1\mathbf{z})\big)$, bottleneck $C \rightarrow C/r \rightarrow C$
3. **Scale:** $x'_c = s_c\,x_c$

Params $\approx 2C^2/r$ (e.g. $r = 16$). Plug after a residual conv. This is **channel attention** with a global descriptor.

Spatial attention (standalone)

Pool over channels (avg and/or max) $\to H \times W \times 2$ (or 1) $\to$ a small conv (often 7×7) $\to$ sigmoid map $M(i, j)$.

$$X' \quad = \quad M \odot X$$

Highlights "where" the object is. Weak alone; strong as a complement to channel attention (next: CBAM).

CBAM — Convolutional Block Attention Module

Woo et al.: **channel then spatial** (order matters empirically).

$$\mathbf{F}' = M_c(\mathbf{F}) \otimes \mathbf{F}, \qquad \mathbf{F}'' = M_s(\mathbf{F}') \otimes \mathbf{F}'$$

M_c: SE-style (avg+max pool $\to$ shared MLP). M_s: avg+max over channels $\to 7 \times 7$ conv.

Drop-in after conv in ResNet. Slightly more FLOPs than SE; usually a bit better localization because of the spatial map.

Coordinate Attention

Hou et al.: SE's GAP **collapses** H and W — loses "where." Coordinate attention factorizes:

$$\text{pool over } W \to (C \times H \times 1), \quad \text{pool over } H \to (C \times 1 \times W)$$

Concat $\to 1 \times 1 \to$ split two sigmoids $\to$ multiply both maps onto X.

Encodes **direction-aware** location (row + column) with near-SE cost. Used in mobile detectors.

Attention vs conv — when to use which

Start with SE on a ResNet. Add CBAM/Coord. if location matters. Full attention is a different architecture, not a drop-in 3×3.

Design a CNN — CIFAR-10 sketch

Problem: 32×32 RGB, 10 classes, small data. A reasonable student net:

Block	Layer	Output shape	Notes
Stem	Conv 3×3, 32, $p = 1$ + ReLU	$32 \times 32 \times 32$	keep size
	MaxPool 2×2	$16 \times 16 \times 32$	
B2	Conv 3×3, 64, $p = 1$ + ReLU $\times 2$	$16 \times 16 \times 64$	
	MaxPool 2×2	$8 \times 8 \times 64$	
B3	Conv 3×3, 128, $p = 1$ + ReLU $\times 2$	$8 \times 8 \times 128$	
Head	GAP	128	**no big FC**
	Linear $128 \to 10$	10 logits	

Params: dominated by last convs, not by a $8 \cdot 8 \cdot 128 \to 512$ flatten. Use BN, dropout, data aug.

Degradation problem — why ResNet

Plain deep nets: training error can **increase** with depth (not just overfitting — the optimizer fails to fit identity-like maps).

Residual block:

$$\mathbf{y} \quad = \quad \mathcal{F}(\mathbf{x}, \{W_i\}) \quad + \quad \mathbf{x}$$

$\mathcal{F}$ learns a **residual**. If identity is best, set $\mathcal{F} \approx 0$ (easier than fitting identity through many nonlinear layers).

- Gradients flow on the skip (additive path).
- Enables 50–152 layers with SGD.
- Bottleneck ($1 \times 1 \to 3 \times 3 \to 1 \times 1$) cuts FLOPs in ResNet-50+.

Quantitative comparison (ImageNet-1k, typical)

How to read the table. Accuracy without params/FLOPs is incomplete. VGG is heavy; ResNet is a better accuracy/compute point; MobileNet trades a little accuracy for tiny FLOPs.

Modify an architecture — experimental justification

Hypothesis. On CIFAR-10, replace the last 3×3, 128-ch conv by **depthwise separable** (3×3 dw $+ 1 \times 1$ pw) to cut params with little accuracy drop.

Control. Same training recipe (epochs, aug, LR). Report mean$\pm$std over 3 seeds.

Variant	Params	Train time	Test acc
Baseline (design slide)	P_0	T_0	A_0
Separable last block	$P_1 < P_0$	T_1	A_1

Accept if $A_0 - A_1$ is small (e.g. < 0.5 pp) and P_1/P_0 drops (e.g. 20%+). **Reject** if accuracy collapses. Always name the **ablation** (only one change).

Takeaways

1. CNN = local MLP + **shared** kernel; libraries implement **cross-correlation**.
2. Images stay as $H \times W \times C$; do not flatten until GAP.
3. Shape: $\lfloor (H + 2p - k)/s \rfloor + 1$; $5 \times 5 \star 3 \times 3 \to 3 \times 3$, $6 \times 6 \to 4 \times 4$; pool 2×2 s=2 halves even maps.
4. Far fewer weights than an MLP because of sparsity + tying; GAP removes giant FC.
5. Backprop: ∂K correlates X with δ; ∂X correlates δ with flipped K.
6. RF grows with depth (dilate / large-kernel to grow faster). Hierarchy: edge $\to$ part $\to$ object. ResNet: $\mathbf{y} = \mathcal{F}(\mathbf{x}) + \mathbf{x}$.
7. Factor convs: DW + PW, groups, 1×1. Recalibrate with SE / CBAM / coordinate attention.